

```

1  // Code your testbench here
2  // or browse Examples
3  module ALU (
4      input  wire      clk,
5      input  wire      reset,
6      input  wire      en_fir,
7
8      input  wire [15:0] data_memory_in_L,
9      input  wire [15:0] data_memory_in_R,
10     input  wire [7:0]  rj_L_data,
11     input  wire [7:0]  rj_R_data,
12     input  wire [15:0] coeff_L_data,
13     input  wire [15:0] coeff_R_data,
14
15     output wire [39:0] shift_outL,
16     output wire [39:0] shift_outR,
17     output wire      done_shifting
18 );
19
20     wire [23:0] sign_ext_L, sign_ext_R;
21     wire [23:0] adder_out_L, adder_out_R;
22
23     wire clear, clear_reg, load_adder, load_shift, shift;
24     wire [3:0] read_addr_rj_L, read_addr_rj_R;
25     wire [4:0] read_addr_coeff_L, read_addr_coeff_R;
26     wire [7:0] read_addr_data_L, read_addr_data_R;
27
28     // Sign Extension
29     SignExtender sign_ext (
30         .datain({data_memory_in_L, data_memory_in_R}),
31         .dataout_L(sign_ext_L),
32         .dataout_R(sign_ext_R)
33     );
34
35     // Adder
36     Adder adder (
37         .dataout_L(sign_ext_L),
38         .shift_outL(shift_outL[39:16]),
39         .add_sub_L(1'b0),
40         .adder_outL(adder_out_L),
41         .dataout_R(sign_ext_R),
42         .shift_outR(shift_outR[39:16]),
43         .add_sub_R(1'b0),
44         .adder_outR(adder_out_R)
45     );
46
47     // Shift Register
48     ShiftRightRegister shift_reg (
49         .clk(clk),
50         .reset(reset),
51         .load(load_shift),
52         .adder_outL(adder_out_L),
53         .adder_outR(adder_out_R),
54         .shift_outL(shift_outL),
55         .shift_outR(shift_outR),
56         .done_shifting(done_shifting)
57     );
58
59     // Controller
60     ALU_Controller controller (
61         .clk(clk),
62         .reset(reset),
63         .en_fir(en_fir),
64         .rj_L_data(rj_L_data),
65         .rj_R_data(rj_R_data),
66         .coeff_L_data(coeff_L_data),
67         .coeff_R_data(coeff_R_data),
68         .clear(clear),
69         .clear_reg(clear_reg),

```

```

70         .load_adder(load_adder),
71         .load_shift(load_shift),
72         .shift(shift),
73         .read_addr_data_L(read_addr_data_L),
74         .read_addr_data_R(read_addr_data_R),
75         .read_addr_rj_L(read_addr_rj_L),
76         .read_addr_rj_R(read_addr_rj_R),
77         .read_addr_coeff_L(read_addr_coeff_L),
78         .read_addr_coeff_R(read_addr_coeff_R),
79         .done_shifting(done_shifting)
80     );
81
82 endmodule
83
84 module ALU_Controller (
85     input wire      clk,
86     input wire      reset,
87     input wire      en_fir,
88     input wire [7:0] rj_L_data,
89     input wire [7:0] rj_R_data,
90     input wire [15:0] coeff_L_data,
91     input wire [15:0] coeff_R_data,
92     input wire      done_shifting,
93
94     output reg      clear,
95     output reg      clear_reg,
96     output reg      load_adder,
97     output reg      load_shift,
98     output reg      shift,
99
100    output reg [7:0] read_addr_data_L,
101    output reg [7:0] read_addr_data_R,
102    output reg [3:0] read_addr_rj_L,
103    output reg [3:0] read_addr_rj_R,
104    output reg [4:0] read_addr_coeff_L,
105    output reg [4:0] read_addr_coeff_R
106 );
107
108 // FSM state definitions
109 reg [2:0] state, next_state;
110 localparam IDLE          = 3'd0,
111             LOAD_RJ       = 3'd1,
112             LOAD_COEFF    = 3'd2,
113             CALC_INDEX    = 3'd3,
114             SHIFT_DATA    = 3'd4,
115             WAIT_SHIFT_DONE = 3'd5,
116             INCREMENT     = 3'd6,
117             DONE          = 3'd7;
118
119 // Internal registers
120 reg [9:0] n_L, n_R;
121 reg [3:0] tap_L, tap_R;
122 reg [4:0] coeff_L_count, coeff_R_count;
123 reg [7:0] rj_L_value, rj_R_value;
124 reg [7:0] coeff_delay_L, coeff_delay_R;
125 reg signed [8:0] index_L, index_R;
126
127 // Sequential logic: FSM state and counters
128 always @(posedge clk or posedge reset) begin
129     if (reset) begin
130         state <= IDLE;
131         n_L <= 0; n_R <= 0;
132         tap_L <= 0; tap_R <= 0;
133         coeff_L_count <= 0; coeff_R_count <= 0;
134         rj_L_value <= 0; rj_R_value <= 0;
135         coeff_delay_L <= 0; coeff_delay_R <= 0;
136         index_L <= 0; index_R <= 0;
137     end else begin
138         state <= next_state;

```

```

139
140     case (state)
141     LOAD_RJ: begin
142         rj_L_value <= rj_L_data;
143         rj_R_value <= rj_R_data;
144     end
145
146     LOAD_COEFF: begin
147         coeff_delay_L <= coeff_L_data[7:0];
148         coeff_delay_R <= coeff_R_data[7:0];
149     end
150
151     CALC_INDEX: begin
152         index_L <= n_L - coeff_delay_L;
153         index_R <= n_R - coeff_delay_R;
154     end
155
156     INCREMENT: begin
157         // LEFT loop logic
158         if (coeff_L_count < rj_L_value - 1)
159             coeff_L_count <= coeff_L_count + 1;
160         else begin
161             coeff_L_count <= 0;
162             if (tap_L < 15)
163                 tap_L <= tap_L + 1;
164             else begin
165                 tap_L <= 0;
166                 if (n_L < 999)
167                     n_L <= n_L + 1;
168             end
169         end
170
171         // RIGHT loop logic
172         if (coeff_R_count < rj_R_value - 1)
173             coeff_R_count <= coeff_R_count + 1;
174         else begin
175             coeff_R_count <= 0;
176             if (tap_R < 15)
177                 tap_R <= tap_R + 1;
178             else begin
179                 tap_R <= 0;
180                 if (n_R < 999)
181                     n_R <= n_R + 1;
182             end
183         end
184     end
185 endcase
186 end
187
188 // Combinational FSM logic
189 always @(*) begin
190     // Default outputs
191     clear = 0;
192     clear_reg = 0;
193     load_adder = 0;
194     load_shift = 0;
195     shift = 0;
196
197     read_addr_rj_L = tap_L;
198     read_addr_rj_R = tap_R;
199     read_addr_coeff_L = coeff_L_count;
200     read_addr_coeff_R = coeff_R_count;
201     read_addr_data_L = 8'd0;
202     read_addr_data_R = 8'd0;
203
204     next_state = state;
205
206     case (state)
207

```

```

208         IDLE: begin
209             if (en_fir) begin
210                 clear = 1;
211                 clear_reg = 1;
212                 next_state = LOAD_RJ;
213             end
214         end
215
216         LOAD_RJ: begin
217             next_state = LOAD_COEFF;
218         end
219
220         LOAD_COEFF: begin
221             next_state = CALC_INDEX;
222         end
223
224         CALC_INDEX: begin
225             load_adder = 1;
226             read_addr_data_L = index_L[7:0];
227             read_addr_data_R = index_R[7:0];
228             next_state = SHIFT_DATA;
229         end
230
231         SHIFT_DATA: begin
232             load_shift = 1;
233             next_state = WAIT_SHIFT_DONE;
234         end
235
236         WAIT_SHIFT_DONE: begin
237             if (done_shifting)
238                 next_state = INCREMENT;
239         end
240
241         INCREMENT: begin
242             if (n_L >= 999 && n_R >= 999)
243                 next_state = DONE;
244             else
245                 next_state = LOAD_RJ;
246             end
247
248         DONE: begin
249             next_state = IDLE;
250         end
251     endcase
252 end
253
254 endmodule
255
256
257
258 module SignExtender (
259     input wire [31:0] datain,
260     output wire [23:0] dataout_L,
261     output wire [23:0] dataout_R
262 );
263     // Sign-extend left half (bits 31:16) using bit 31
264     assign dataout_L = {{8{datain[31]}}, datain[31:16]};
265
266     // Sign-extend right half (bits 15:0) using bit 15
267     assign dataout_R = {{8{datain[15]}}, datain[15:0]};
268 endmodule
269
270
271 //adder is been updated to do add on both Left and Right
272 module Adder (
273     //Left side of the adder
274     input wire [23:0] dataout_L,
275     input wire [23:0] shift_outL,
276     input wire          add_sub_L, // 0: add, 1: subtract

```

```

277     output wire [23:0] adder_outL,
278
279     //Right side of the adder
280     input wire [23:0] dataout_R,
281     input wire [23:0] shift_outR,
282     input wire      add_sub_R, // 0: add, 1: subtract
283     output wire [23:0] adder_outR
284 );
285     assign adder_outL = add_sub_L ? (shift_outL - dataout_L) : (shift_outL + dataout_L);
286     assign adder_outR = add_sub_R ? (shift_outR - dataout_R) : (shift_outR + dataout_R);
287 endmodule
288
289
290
291 module ShiftRightRegister (
292     input wire      clk,
293     input wire      reset,
294     input wire      load,
295     input wire [23:0] adder_outL,
296     input wire [23:0] adder_outR,
297     output reg [39:0] shift_outL,
298     output reg [39:0] shift_outR,
299     output reg      done_shifting
300 );
301
302     reg [5:0] shift_count;
303     reg      shifting;
304
305     always @(posedge clk or posedge reset) begin
306         if (reset) begin
307             shift_outL      <= 40'd0;
308             shift_outR      <= 40'd0;
309             shift_count     <= 6'd0;
310             shifting        <= 1'b0;
311             done_shifting   <= 1'b0;
312         end else begin
313             done_shifting <= 1'b0;
314
315             if (load) begin
316                 shift_outL[39:16] <= adder_outL;
317                 shift_outL[15:0]  <= 16'd0;
318                 shift_outR[39:16] <= adder_outR;
319                 shift_outR[15:0]  <= 16'd0;
320                 shift_count      <= 6'd0;
321                 shifting         <= 1'b1;
322             end else if (shifting) begin
323                 shift_outL <= {1'b0, shift_outL[39:1]};
324                 shift_outR <= {1'b0, shift_outR[39:1]};
325                 shift_count <= shift_count + 1;
326
327                 if (shift_count == 6'd31) begin
328                     shifting      <= 1'b0;
329                     done_shifting <= 1'b1;
330                 end
331             end
332         end
333     end
334
335 endmodule
336
337
338 //Testbench for ALU_Top Module
339 // Code your testbench here
340 // or browse Examples
341 module ALU (
342     input wire      clk,
343     input wire      reset,
344     input wire      en_fir,
345

```

```

346     input wire [15:0] data_memory_in_L,
347     input wire [15:0] data_memory_in_R,
348     input wire [7:0]  rj_L_data,
349     input wire [7:0]  rj_R_data,
350     input wire [15:0] coeff_L_data,
351     input wire [15:0] coeff_R_data,
352
353     output wire [39:0] shift_outL,
354     output wire [39:0] shift_outR,
355     output wire      done_shifting
356 );
357
358     wire [23:0] sign_ext_L, sign_ext_R;
359     wire [23:0] adder_out_L, adder_out_R;
360
361     wire clear, clear_reg, load_adder, load_shift, shift;
362     wire [3:0] read_addr_rj_L, read_addr_rj_R;
363     wire [4:0] read_addr_coeff_L, read_addr_coeff_R;
364     wire [7:0] read_addr_data_L, read_addr_data_R;
365
366     // Sign Extension
367     SignExtender sign_ext (
368         .datain({data_memory_in_L, data_memory_in_R}),
369         .dataout_L(sign_ext_L),
370         .dataout_R(sign_ext_R)
371     );
372
373     // Adder
374     Adder adder (
375         .dataout_L(sign_ext_L),
376         .shift_outL(shift_outL[39:16]),
377         .add_sub_L(1'b0),
378         .adder_outL(adder_out_L),
379         .dataout_R(sign_ext_R),
380         .shift_outR(shift_outR[39:16]),
381         .add_sub_R(1'b0),
382         .adder_outR(adder_out_R)
383     );
384
385     // Shift Register
386     ShiftRightRegister shift_reg (
387         .clk(clk),
388         .reset(reset),
389         .load(load_shift),
390         .adder_outL(adder_out_L),
391         .adder_outR(adder_out_R),
392         .shift_outL(shift_outL),
393         .shift_outR(shift_outR),
394         .done_shifting(done_shifting)
395     );
396
397     // Controller
398     ALU_Controller controller (
399         .clk(clk),
400         .reset(reset),
401         .en_fir(en_fir),
402         .rj_L_data(rj_L_data),
403         .rj_R_data(rj_R_data),
404         .coeff_L_data(coeff_L_data),
405         .coeff_R_data(coeff_R_data),
406         .clear(clear),
407         .clear_reg(clear_reg),
408         .load_adder(load_adder),
409         .load_shift(load_shift),
410         .shift(shift),
411         .read_addr_data_L(read_addr_data_L),
412         .read_addr_data_R(read_addr_data_R),
413         .read_addr_rj_L(read_addr_rj_L),
414         .read_addr_rj_R(read_addr_rj_R),

```

```

415         .read_addr_coeff_L(read_addr_coeff_L),
416         .read_addr_coeff_R(read_addr_coeff_R),
417         .done_shifting(done_shifting)
418     );
419
420 endmodule
421
422 module ALU_Controller (
423     input wire      clk,
424     input wire      reset,
425     input wire      en_fir,
426     input wire [7:0] rj_L_data,
427     input wire [7:0] rj_R_data,
428     input wire [15:0] coeff_L_data,
429     input wire [15:0] coeff_R_data,
430     input wire      done_shifting,
431
432     output reg       clear,
433     output reg       clear_reg,
434     output reg       load_adder,
435     output reg       load_shift,
436     output reg       shift,
437
438     output reg [7:0]  read_addr_data_L,
439     output reg [7:0]  read_addr_data_R,
440     output reg [3:0]  read_addr_rj_L,
441     output reg [3:0]  read_addr_rj_R,
442     output reg [4:0]  read_addr_coeff_L,
443     output reg [4:0]  read_addr_coeff_R
444 );
445
446 // FSM state definitions
447 reg [2:0] state, next_state;
448 localparam IDLE          = 3'd0,
449             LOAD_RJ       = 3'd1,
450             LOAD_COEFF    = 3'd2,
451             CALC_INDEX    = 3'd3,
452             SHIFT_DATA    = 3'd4,
453             WAIT_SHIFT_DONE = 3'd5,
454             INCREMENT     = 3'd6,
455             DONE          = 3'd7;
456
457 // Internal registers
458 reg [9:0] n_L, n_R;
459 reg [3:0] tap_L, tap_R;
460 reg [4:0] coeff_L_count, coeff_R_count;
461 reg [7:0] rj_L_value, rj_R_value;
462 reg [7:0] coeff_delay_L, coeff_delay_R;
463 reg signed [8:0] index_L, index_R;
464
465 // Sequential logic: FSM state and counters
466 always @(posedge clk or posedge reset) begin
467     if (reset) begin
468         state <= IDLE;
469         n_L <= 0; n_R <= 0;
470         tap_L <= 0; tap_R <= 0;
471         coeff_L_count <= 0; coeff_R_count <= 0;
472         rj_L_value <= 0; rj_R_value <= 0;
473         coeff_delay_L <= 0; coeff_delay_R <= 0;
474         index_L <= 0; index_R <= 0;
475     end else begin
476         state <= next_state;
477
478         case (state)
479             LOAD_RJ: begin
480                 rj_L_value <= rj_L_data;
481                 rj_R_value <= rj_R_data;
482             end
483

```

```

484     LOAD_COEFF: begin
485         coeff_delay_L <= coeff_L_data[7:0];
486         coeff_delay_R <= coeff_R_data[7:0];
487     end
488
489     CALC_INDEX: begin
490         index_L <= n_L - coeff_delay_L;
491         index_R <= n_R - coeff_delay_R;
492     end
493
494     INCREMENT: begin
495         // LEFT loop logic
496         if (coeff_L_count < rj_L_value - 1)
497             coeff_L_count <= coeff_L_count + 1;
498         else begin
499             coeff_L_count <= 0;
500             if (tap_L < 15)
501                 tap_L <= tap_L + 1;
502             else begin
503                 tap_L <= 0;
504                 if (n_L < 999)
505                     n_L <= n_L + 1;
506             end
507         end
508
509         // RIGHT loop logic
510         if (coeff_R_count < rj_R_value - 1)
511             coeff_R_count <= coeff_R_count + 1;
512         else begin
513             coeff_R_count <= 0;
514             if (tap_R < 15)
515                 tap_R <= tap_R + 1;
516             else begin
517                 tap_R <= 0;
518                 if (n_R < 999)
519                     n_R <= n_R + 1;
520             end
521         end
522     end
523 endcase
524 end
525 end
526
527 // Combinational FSM logic
528 always @(*) begin
529     // Default outputs
530     clear = 0;
531     clear_reg = 0;
532     load_adder = 0;
533     load_shift = 0;
534     shift = 0;
535
536     read_addr_rj_L = tap_L;
537     read_addr_rj_R = tap_R;
538     read_addr_coeff_L = coeff_L_count;
539     read_addr_coeff_R = coeff_R_count;
540     read_addr_data_L = 8'd0;
541     read_addr_data_R = 8'd0;
542
543     next_state = state;
544
545     case (state)
546     IDLE: begin
547         if (en_fir) begin
548             clear = 1;
549             clear_reg = 1;
550             next_state = LOAD_RJ;
551         end
552     end

```



```

553
554     LOAD_RJ: begin
555         next_state = LOAD_COEFF;
556     end
557
558     LOAD_COEFF: begin
559         next_state = CALC_INDEX;
560     end
561
562     CALC_INDEX: begin
563         load_adder = 1;
564         read_addr_data_L = index_L[7:0];
565         read_addr_data_R = index_R[7:0];
566         next_state = SHIFT_DATA;
567     end
568
569     SHIFT_DATA: begin
570         load_shift = 1;
571         next_state = WAIT_SHIFT_DONE;
572     end
573
574     WAIT_SHIFT_DONE: begin
575         if (done_shifting)
576             next_state = INCREMENT;
577     end
578
579     INCREMENT: begin
580         if (n_L >= 999 && n_R >= 999)
581             next_state = DONE;
582         else
583             next_state = LOAD_RJ;
584         end
585
586     DONE: begin
587         next_state = IDLE;
588     end
589 endcase
590 end
591
592 endmodule
593
594
595
596 module SignExtender (
597     input wire [31:0] datain,
598     output wire [23:0] dataout_L,
599     output wire [23:0] dataout_R
600 );
601     // Sign-extend left half (bits 31:16) using bit 31
602     assign dataout_L = {{8{datain[31]}}, datain[31:16]};
603
604     // Sign-extend right half (bits 15:0) using bit 15
605     assign dataout_R = {{8{datain[15]}}, datain[15:0]};
606 endmodule
607
608
609 //adder is been updated to do add on both Left and Right
610 module Adder (
611     //Left side of the adder
612     input wire [23:0] dataout_L,
613     input wire [23:0] shift_outL,
614     input wire      add_sub_L, // 0: add, 1: subtract
615     output wire [23:0] adder_outL,
616
617     //Right side of the adder
618     input wire [23:0] dataout_R,
619     input wire [23:0] shift_outR,
620     input wire      add_sub_R, // 0: add, 1: subtract
621     output wire [23:0] adder_outR

```

```

622 );
623     assign adder_outL = add_sub_L ? (shift_outL - dataout_L) : (shift_outL + dataout_L);
624     assign adder_outR = add_sub_R ? (shift_outR - dataout_R) : (shift_outR + dataout_R);
625 endmodule
626
627
628
629 module ShiftRightRegister (
630     input wire      clk,
631     input wire      reset,
632     input wire      load,
633     input wire [23:0] adder_outL,
634     input wire [23:0] adder_outR,
635     output reg [39:0] shift_outL,
636     output reg [39:0] shift_outR,
637     output reg      done_shifting
638 );
639
640     reg [5:0] shift_count;
641     reg      shifting;
642
643     always @(posedge clk or posedge reset) begin
644         if (reset) begin
645             shift_outL    <= 40'd0;
646             shift_outR    <= 40'd0;
647             shift_count   <= 6'd0;
648             shifting      <= 1'b0;
649             done_shifting <= 1'b0;
650         end else begin
651             done_shifting <= 1'b0;
652
653             if (load) begin
654                 shift_outL[39:16] <= adder_outL;
655                 shift_outL[15:0]  <= 16'd0;
656                 shift_outR[39:16] <= adder_outR;
657                 shift_outR[15:0]  <= 16'd0;
658                 shift_count       <= 6'd0;
659                 shifting          <= 1'b1;
660             end else if (shifting) begin
661                 shift_outL <= {1'b0, shift_outL[39:1]};
662                 shift_outR <= {1'b0, shift_outR[39:1]};
663                 shift_count <= shift_count + 1;
664
665                 if (shift_count == 6'd31) begin
666                     shifting      <= 1'b0;
667                     done_shifting <= 1'b1;
668                 end
669             end
670         end
671     end
672
673 endmodule
674

```